

Phase 1: Test-Driven Development and Unit Testing

Objectives

The objective of phase 1 is to apply the following techniques and tools in a practical case study:

- Interface design,
- Test-Driven Development,
- Functional test design, and
- Unit Testing and JUnit tool.

Description of the Unit

The first phase of this project concerns test-driven development of a module that has the basic functionalities regarding an autonomous parking assistance system as follows:

- We assume that the car moving along a perfectly straight street which is 500 meters long and registers the available parking places on its right-hand side.
- To do this the car moves forward and uses two ultrasound sensors to check whether there is a free space on its right hand side.
- The measurements are combined and filtered to reliably find a free parking stretch of 5 meters.
- The car then moves to the end of the 5 meter stretch and runs a standard parallel reverse parking maneuver.

To do this, first design a (Java) class with the following methods:

- **MoveForward**: This method moves the car 1 meter forward, queries the two sensors through the **isEmpty** method described below and returns a data structure that contains the current position of the car, and the situation of the detected parking places up to now. The car cannot be moved forward beyond the end of the street.
- **isEmpty**: This method queries the two ultrasound sensors at least 5 times and filters the noise in their results and returns the distance in cm to the nearest object in the right hand side. If one sensor is detected to continuously return very noisy output, it should be completely disregarded. You can use averaging or any other statistical method to filter the noise from the signals received from the ultrasound sensors.
- **MoveBackward**: The same as **MoveForward** above; only it moves the car 1 meter backwards. The car cannot be moved behind if it is already at the beginning of the street.
- **Park**: It performs a pre-programmed reverse parallel parking maneuver, if it is already positioned at an empty parking space stretch, or moves the car forwards towards the end of the street until such a stretch is detected, and then parks it.
- **UnPark**: It moves the car forward (and to left) to front of the parking place, if it is parked.

- **WhereIs:** This method returns the current position of the car in the street as well as its (un)parked status.

In all of the movements, the car sends a signal to the actuators moving the car, but the actuators are not modeled in this phase. Likewise, all the sensor signals come from sensor classes that are not modeled in this phase. You can assume random or fixed sensor inputs in this phase. The sensor inputs (for a properly working sensor) are integers (in the range of 0 to 200) indicating the distance in cm to the nearest object on the right.

Deliverables

There are two main deliverables for this phase: a single PDF file documenting the outcome of each and every of the following steps and a ZIP file containing the source code of the software implemented as the final outcome of this phase. **The deadline for submitting this phase deliverables is September 18, 2026.**

Part 1: Interface and Test Design

Before starting any implementation, think of the data structures you need for this unit and a short description of the methods you need and what they should do. Think if any of the methods need any parameters, what is their exact return type. For the whole class, consider what should be its internal state, that is, what current information needs to be available at all times for the class to function as expected.

The next part of this deliverable concerns the interface and the test design of the above-specified module. For the interface, you need to specify the signature of the methods: name, input argument types, and output return type. For each interface method, you need to give its specification in the following form:

```
/**  
  Description  
  Pre-condition:  
  Post-condition:  
  Test-cases:  
*/
```

To design your tests, first use one of the functional testing methods (preferably: classification tree or decision table) to partition the domain of different inputs (or output) using the above-given requirements. Then define a test suite (a set of test-case) with concrete input values and expected outputs.

Your deliverables will be judged based on:

- **soundness:** whether the interfaces have been correctly specified (according to the requirements), the test technique has been correctly exploited to design test cases, and
- **completeness:** whether all requirements have been considered and all test-cases necessary to cover them have been given.

Part 2: Test Driven Development

For each and every method, apply the principles of test-driven development to implement the interfaces in order to satisfy each and every test-case. Each line of code should be augmented with the reason why it has been added (which test case it is supposed to satisfy). Before you start each step in the implementation,

implement a test-case as a ***JUnit*** test, observe how it fails, add the line(s) of code necessary to satisfy it, observe that all tests pass afterwards and comment the line(s) of code to specify why they have been added. In your report, describe in a step-wise manner how each piece of implementation has been added to fulfill a test-case.